

Université du Québec à Chicoutimi

Département d'Informatique et de Mathématique

Architecture des applications d'entreprise

8INF853 — Professeur Hamid Mcheick

Session : Été 2026

TP2 — Question 4

**Architecture Microservices :
Conception et implémentation d'une
application bancaire distribuée**

Thomas ESTEVES

Julien LARZUL

2 juin 2026

Table des matières

1	Introduction	3
1.1	Mise en contexte	3
1.2	Objectif du travail	3
1.3	Plan du document	3
2	Partie A — Rôle de l'architecture Microservices	4
2.1	Définition et origine	4
2.1.1	Qu'est-ce qu'une architecture microservices ?	4
2.1.2	Contexte historique	4
2.2	Principes fondamentaux	4
2.2.1	Décomposition par domaine métier	4
2.2.2	Indépendance des services	5
2.2.3	Communication via API REST	5
2.2.4	Déploiement indépendant	5
2.2.5	Décentralisation des données	5
2.3	Étapes du cycle de vie	5
2.3.1	Conception et découpage en services	5
2.3.2	Développement indépendant	5
2.3.3	Conteneurisation avec Docker	6
2.3.4	Orchestration et déploiement	6
2.3.5	Monitoring et observabilité	6
2.4	Comparaison Monolithe vs Microservices	6
3	Partie B — Application bancaire en microservices	7
3.1	Présentation de l'application	7
3.1.1	Description générale	7
3.1.2	Choix technologiques justifiés	7
3.2	Identification et description des microservices	8
3.2.1	Discovery Service (Eureka Server) — Port 8761	8
3.2.2	API Gateway (Spring Cloud Gateway) — Port 8080	8
3.2.3	Auth Service — Port 8081	8
3.2.4	Account Service — Port 8082	9
3.2.5	Transaction Service — Port 8083	9
3.2.6	Notification Service — Port 8084	10
3.3	Diagrammes	10
3.3.1	Diagramme d'architecture globale	10
3.3.2	Diagramme de séquence : flux d'authentification	11
3.3.3	Diagramme de séquence : flux d'un virement	12

3.3.4	Diagramme de séquence : flux de notification	13
3.4	Implémentation	14
3.4.1	Discovery Service	14
3.4.2	API Gateway	15
3.4.3	Auth Service	16
3.4.4	Account Service	18
3.4.5	Transaction Service	20
3.4.6	Notification Service	22
3.4.7	Conteneurisation — Dockerfile	23
3.4.8	Orchestration — Docker Compose	23
3.4.9	Instructions de lancement	25
4	Partie C — Avantages et inconvénients	25
4.1	Avantages	25
4.1.1	Scalabilité indépendante	25
4.1.2	Déploiement continu facilité	26
4.1.3	Isolation des pannes	26
4.1.4	Flexibilité technologique	26
4.1.5	Équipes autonomes	26
4.1.6	Maintenabilité et compréhensibilité	26
4.2	Inconvénients	27
4.2.1	Complexité opérationnelle	27
4.2.2	Latence réseau inter-services	27
4.2.3	Gestion de la cohérence des données	27
4.2.4	Coût d'infrastructure	27
4.2.5	Débogage difficile	27
4.2.6	Surcharge pour les petits projets	27
4.3	Quand utiliser les microservices ?	28
4.3.1	Cas favorables	28
4.3.2	Cas défavorables	28
4.3.3	Recommandations	28
5	Conclusion	29
6	Références bibliographiques	30

1 Introduction

1.1 Mise en contexte

Dans le paysage actuel du développement logiciel, les entreprises font face à des exigences croissantes en matière de scalabilité, de résilience et de rapidité de mise en marché. L'architecture monolithique traditionnelle, bien qu'adaptée aux petits projets, atteint rapidement ses limites lorsque les applications gagnent en complexité et en nombre d'utilisateurs. C'est dans ce contexte qu'émerge l'architecture microservices, un paradigme architectural qui propose de décomposer une application en un ensemble de services autonomes, chacun responsable d'un domaine métier spécifique.

L'architecture microservices s'inscrit pleinement dans la philosophie DevOps, qui prône l'automatisation de toutes les étapes du cycle de vie d'un logiciel, de la conception au déploiement en passant par les tests et la surveillance. Cette synergie entre microservices et DevOps constitue aujourd'hui l'une des approches les plus adoptées par les grandes organisations technologiques telles que Netflix, Amazon et Spotify [2].

1.2 Objectif du travail

Ce travail pratique vise à approfondir notre compréhension de l'architecture microservices dans le cadre du cours *Architecture des applications d'entreprise* (8INF853). Plus spécifiquement, nos objectifs sont :

1. **Expliquer le rôle** de l'architecture microservices et de la méthode DevOps dans le développement logiciel moderne ;
2. **Concevoir et implémenter** une application bancaire distribuée en utilisant cette architecture, en identifiant les microservices, en produisant les diagrammes appropriés et en fournissant les implémentations correspondantes ;
3. **Analyser les avantages et les inconvénients** de cette approche architecturale de manière critique et argumentée.

1.3 Plan du document

Le présent rapport est organisé en trois parties principales. La **Partie A** (section 2) présente le rôle et les principes fondamentaux de l'architecture microservices. La **Partie B** (section 3) détaille la conception et l'implémentation de notre application bancaire. Enfin, la **Partie C** (section 4) propose une analyse critique des avantages et inconvénients de cette approche.

2 Partie A — Rôle de l'architecture Microservices

2.1 Définition et origine

2.1.1 Qu'est-ce qu'une architecture microservices ?

L'architecture microservices est un style architectural qui structure une application comme un ensemble de services faiblement couplés, organisés autour de capacités métier (*business capabilities*). Chaque microservice est un processus indépendant qui communique avec les autres via des mécanismes légers, généralement des API REST sur le protocole HTTP [1].

Selon Martin Fowler, les caractéristiques fondamentales d'une architecture microservices incluent : la componentisation via des services, l'organisation autour des capacités métier, la décentralisation de la gouvernance et des données, ainsi que la conception orientée vers la tolérance aux pannes [1].

2.1.2 Contexte historique

Le passage du monolithe aux microservices s'est effectué progressivement au cours des deux dernières décennies :

- **Années 2000** : Dominance des architectures monolithiques et des architectures orientées services (SOA) utilisant des protocoles lourds comme SOAP et des bus de services d'entreprise (ESB).
- **2005–2010** : Émergence des concepts de services légers avec REST. Amazon et Netflix commencent à migrer vers des architectures distribuées pour répondre à leurs besoins de scalabilité.
- **2011–2014** : Le terme *microservices* est formalisé. James Lewis et Martin Fowler publient leur article fondateur en 2014 [1]. Les outils comme Docker (2013) facilitent la conteneurisation.
- **2015–aujourd'hui** : Adoption massive grâce à Kubernetes, Spring Cloud, et l'écosystème cloud-native. Les microservices deviennent le standard pour les applications distribuées à grande échelle.

2.2 Principes fondamentaux

2.2.1 Décomposition par domaine métier

Le principe de décomposition par domaine métier, inspiré du *Domain-Driven Design* (DDD) d'Eric Evans [4], consiste à identifier les frontières naturelles du système (*bounded contexts*) et à créer un service pour chacun. Dans notre application bancaire, nous avons identifié quatre domaines métier distincts : l'authentification, la gestion des comptes, les transactions financières et les notifications.

2.2.2 Indépendance des services

Chaque microservice doit pouvoir être développé, testé, déployé et mis à l'échelle de manière indépendante. Cette indépendance se traduit par :

- Un **code source séparé** : chaque service possède son propre projet Maven ;
- Une **base de données propre** (*database per service*) : dans notre cas, chaque service utilise sa propre instance H2 ;
- Un **processus de déploiement autonome** : chaque service dispose de son propre .

2.2.3 Communication via API REST

Les microservices communiquent entre eux via des interfaces bien définies, typiquement des API REST. Ce choix favorise un couplage faible entre les services. Dans notre application, le `client` communique avec le `server` et le `gateway` via OpenFeign, un client HTTP déclaratif qui simplifie les appels inter-services [10].

2.2.4 Déploiement indépendant

Grâce à la conteneurisation avec Docker, chaque service peut être déployé indépendamment des autres. Le fichier `docker-compose.yml` orchestre le démarrage de l'ensemble des conteneurs tout en respectant les dépendances de démarrage via les `depends_on` et les `wait_for`.

2.2.5 Décentralisation des données

Contrairement à l'approche monolithique où une seule base de données est partagée, chaque microservice gère sa propre base de données. Ce principe, connu sous le nom de *database per service*, garantit l'autonomie de chaque service et élimine les dépendances de schéma partagé [3].

2.3 Étapes du cycle de vie

2.3.1 Conception et découpage en services

La première étape consiste à analyser le domaine métier et à identifier les services. Cette décomposition s'appuie sur les principes du DDD et sur le principe de responsabilité unique (SRP). Chaque service doit être suffisamment petit pour être maintenu par une équipe réduite, mais suffisamment cohérent pour encapsuler un domaine métier complet.

2.3.2 Développement indépendant

Chaque équipe développe son service de manière autonome, en choisissant les technologies les mieux adaptées. Dans notre cas, tous les services utilisent Java 17 et Spring Boot 3.x, mais le principe permettrait d'utiliser des langages différents pour chaque service.

2.3.3 Conteneurisation avec Docker

La conteneurisation permet d'empaqueter chaque service avec toutes ses dépendances dans une image Docker légère et reproductible. Le multi-étapes (*multi-stage build*) que nous utilisons sépare la phase de compilation (Maven) de la phase d'exécution (JRE Alpine), réduisant ainsi la taille de l'image finale.

2.3.4 Orchestration et déploiement

Docker Compose permet d'orchestrer le déploiement local de l'ensemble des services. En environnement de production, des outils comme Kubernetes prennent le relais pour gérer la scalabilité automatique, la répartition de charge et la tolérance aux pannes.

2.3.5 Monitoring et observabilité

L'observabilité est essentielle dans une architecture distribuée. Chaque service expose des endpoints via Spring Boot Actuator, permettant de surveiller l'état de santé de l'application. Le assure la journalisation des événements métier.

2.4 Comparaison Monolithe vs Microservices

TABLE 1 – Comparaison entre architecture monolithique et microservices

Critère	Monolithe	Microservices
Déploiement	Tout ou rien	Indépendant par service
Scalabilité	Verticale (tout le système)	Horizontale (par service)
Base de données	Unique et partagée	Une par service
Technologies	Uniformes	Hétérogènes possibles
Couplage	Fort (modules liés)	Faible (API REST)
Complexité initiale	Faible	Élevée
Tolérance aux pannes	Panne globale	Isolation des pannes
Taille des équipes	Grande équipe unique	Petites équipes autonomes
Tests	Tests d'intégration simples	Tests distribués complexes
Communication	Appels de méthodes en mémoire	Appels réseau (latence)

3 Partie B — Application bancaire en microservices

3.1 Présentation de l'application

3.1.1 Description générale

Notre application bancaire est un système distribué composé de six microservices indépendants. Elle permet aux utilisateurs de s'inscrire, de s'authentifier, de gérer des comptes bancaires (courant et épargne), d'effectuer des opérations financières (dépôt, retrait, virement) et de consulter l'historique des notifications associées à leurs transactions.

L'architecture suit le modèle classique des microservices avec une passerelle API centralisée (*API Gateway*) et un registre de découverte de services (*Service Registry*).

3.1.2 Choix technologiques justifiés

TABLE 2 – Stack technique et justifications

Technologie	Version	Justification
Java	17 (LTS)	Langage mature, typage fort, écosystème riche pour les applications d'entreprise
Spring Boot	3.x	Framework de référence pour les microservices Java, auto-configuration, démarrage rapide
Spring Cloud	2023.x	Fournit les patrons d'infrastructure distribuée (Gateway, Discovery, Feign)
Netflix Eureka	—	Registre de services éprouvé, intégration native avec Spring Cloud
OpenFeign	—	Client HTTP déclaratif, simplifie la communication inter-services
H2 Database	—	Base de données en mémoire, idéale pour le prototypage et les démonstrations
Docker	—	Conteneurisation standardisée, reproductibilité des environnements
Docker Compose	—	Orchestration multi-conteneurs pour le déploiement local
Maven	—	Gestion des dépendances et du cycle de build
Lombok	—	Réduction du code <i>boilerplate</i> (getters, setters, constructeurs)
BCrypt	—	Hachage sécurisé des mots de passe
JWT	—	Authentification stateless, adaptée aux architectures distribuées

3.2 Identification et description des microservices

3.2.1 Discovery Service (Eureka Server) — Port 8761

Rôle et responsabilité : Le agit comme un registre central où tous les microservices s'enregistrent au démarrage. Il permet la découverte dynamique des services, éliminant le besoin de coder en dur les adresses IP et les ports. Ce patron est connu sous le nom de *Service Registry* [3].

TABLE 3 – Endpoints du discovery-service

Méthode	Endpoint	Description
GET	/eureka/apps	Liste tous les services enregistrés
GET	/actuator/health	État de santé du service

Dépendances : Aucune. C'est le premier service à démarrer.

Base de données : Aucune (registre en mémoire).

3.2.2 API Gateway (Spring Cloud Gateway) — Port 8080

Rôle et responsabilité : L' est le point d'entrée unique de l'application. Il route les requêtes HTTP entrantes vers le microservice approprié en se basant sur les prédicats de chemin (*path predicates*). Ce patron est connu sous le nom de *API Gateway Pattern* [3].

TABLE 4 – Routes configurées dans l'API Gateway

Chemin	Service cible	Description
/auth/**	auth-service	Authentification et inscription
/accounts/**	account-service	Gestion des comptes
/transactions/**	transaction-service	Opérations financières
/notifications/**	notification-service	Consultation des notifications

Dépendances : discovery-service (pour la résolution des adresses).

Base de données : Aucune.

3.2.3 Auth Service — Port 8081

Rôle et responsabilité : Le gère l'authentification des utilisateurs. Il permet l'inscription et la connexion, et génère des jetons JWT (*JSON Web Tokens*) pour l'authentification stateless.

TABLE 5 – Endpoints du auth-service

Méthode	Endpoint	Description
POST	/auth/register	Inscription d'un nouvel utilisateur
POST	/auth/login	Connexion (retourne un JWT)
GET	/auth/health	État de santé du service

Dépendances : discovery-service.

Base de données : (H2) — table .

3.2.4 Account Service — Port 8082

Rôle et responsabilité : Le gère le cycle de vie des comptes bancaires : création, consultation et mise à jour des soldes. Il expose également un endpoint interne pour la mise à jour des soldes, utilisé par le .

TABLE 6 – Endpoints du account-service

Méthode	Endpoint	Description
POST	/accounts	Créer un nouveau compte
GET	/accounts/{id}	Consulter un compte par ID
GET	/accounts/user/{userId}	Comptes d'un utilisateur
PUT	/accounts/{id}/balance	Mettre à jour le solde (interne)

Dépendances : discovery-service.

Base de données : (H2) — table .

3.2.5 Transaction Service — Port 8083

Rôle et responsabilité : Le orchestre les opérations financières : dépôts, retraits et virements. C'est le service le plus interconnecté : il appelle le pour vérifier et mettre à jour les soldes, et le pour journaliser chaque opération.

TABLE 7 – Endpoints du transaction-service

Méthode	Endpoint	Description
POST	/transactions/deposit	Effectuer un dépôt
POST	/transactions/withdraw	Effectuer un retrait
POST	/transactions/transfer	Effectuer un virement
GET	/transactions/{accountId}	Historique des transactions

Dépendances : discovery-service, account-service (via Feign), notification-service (via Feign).

Base de données : (H2) — table .

3.2.6 Notification Service — Port 8084

Rôle et responsabilité : Le reçoit et stocke les notifications générées par les opérations financières. Il assure la traçabilité de toutes les actions effectuées dans le système.

TABLE 8 – Endpoints du notification-service

Méthode	Endpoint	Description
POST	/notifications	Créer une notification (interne)
GET	/notifications	Lister toutes les notifications
GET	/notifications/account/{id}	Notifications par compte

Dépendances : discovery-service.

Base de données : (H2) — table .

3.3 Diagrammes

Les diagrammes suivants illustrent l'architecture de notre application et les flux de communication entre les services. Ils sont présentés ici en notation textuelle Mermaid.js et sont fournis séparément en tant que fichiers rendus.

3.3.1 Diagramme d'architecture globale

La figure 1 présente la vue d'ensemble de l'architecture. Le client interagit exclusivement avec l'API Gateway, qui route les requêtes vers les services appropriés. Tous les services s'enregistrent auprès d'Eureka pour la découverte dynamique.

FIGURE 1 – Diagramme d'architecture globale de l'application bancaire

3.3.2 Diagramme de séquence : flux d'authentification

La figure 2 illustre le flux d'inscription et de connexion d'un utilisateur.

FIGURE 2 – Diagramme de séquence — Flux d'authentification

3.3.3 Diagramme de séquence : flux d'un virement

La figure 3 détaille le processus d'un virement bancaire, qui implique trois microservices.

FIGURE 3 – Diagramme de séquence — Flux d'un virement bancaire

3.3.4 Diagramme de séquence : flux de notification

La figure 4 illustre comment le reçoit et traite les événements.

FIGURE 4 – Diagramme de séquence — Flux de notification

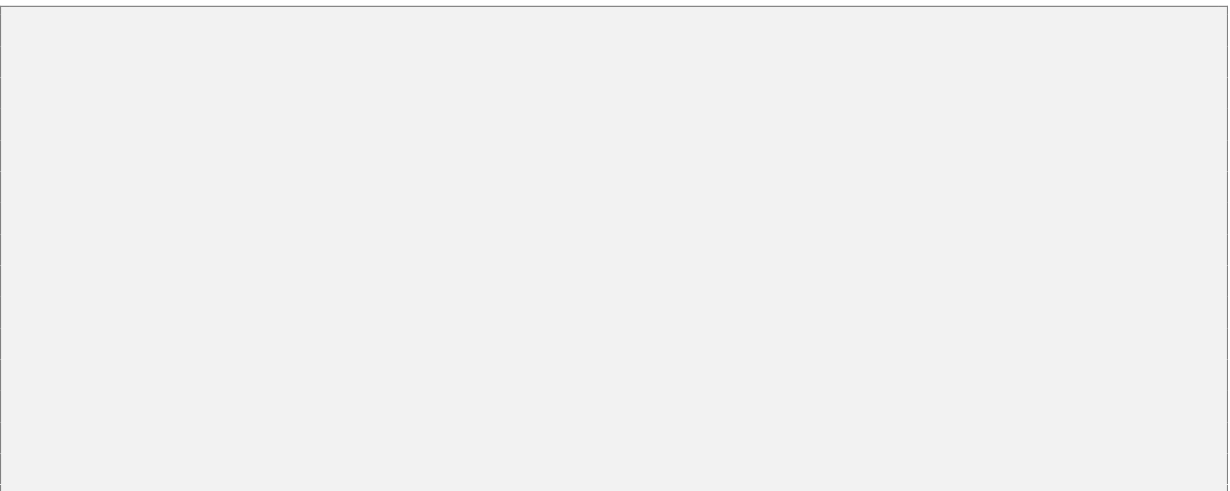
3.4 Implémentation

Cette section présente les extraits de code les plus significatifs de chaque microservice, accompagnés d'explications détaillées.

3.4.1 Discovery Service

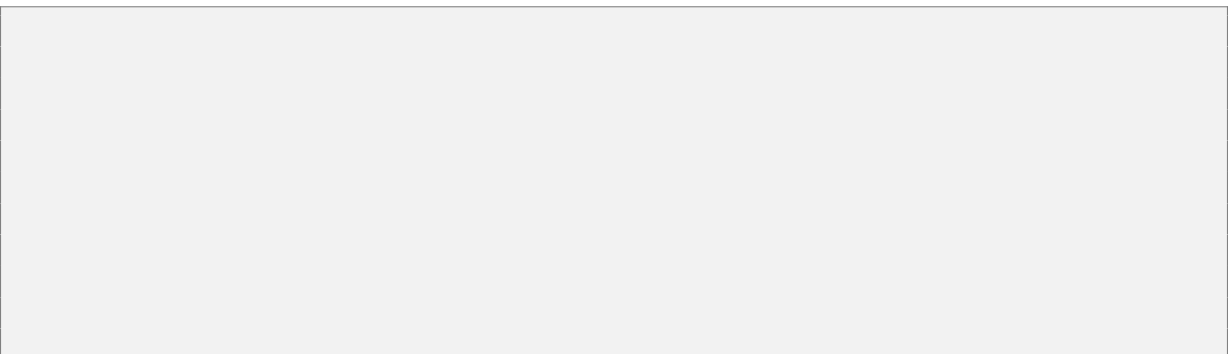
Le est le composant le plus simple : il s'agit d'un serveur Eureka activé par une simple annotation.

```
1
2
3
4
5
6
7
8
9
10
11
12
13
14
15
```



Listing 1 – DiscoveryApplication.java — Point d'entrée du serveur Eureka

```
1
2
3
4
5
6
7
8
9
10
11
```



12
13
14
15
16
17

Listing 2 – application.yml du discovery-service

3.4.2 API Gateway

L'API Gateway utilise Spring Cloud Gateway pour router les requêtes. La configuration est entièrement déclarative dans le fichier YAML.

1
2
3
4
5
6
7
8
9
10
11

Listing 3 – GatewayApplication.java

1
2
3
4
5
6
7
8
9
10
11
12
13
14
15
16
17
18
19
20
21
22
23
24
25
26
27
28
29

30

Listing 4 – application.yml de l'API Gateway — Configuration du routage

3.4.3 Auth Service

Le service d'authentification implémente le patron d'inscription/connexion avec hachage BCrypt et génération de jetons JWT.

Entité User

```
1
2
3
4
5
6
7
8
9
10
11
12
13
14
15
16
17
18
19
20
```

Listing 5 – User.java — Entité JPA représentant un utilisateur

Contrôleur REST

```
1
2
3
4
5
6
7
8
9
10
11
12
13
14
15
16
17
18
19
20
21
22
```

23
24
25
26
27
28

Listing 6 – AuthController.java — Endpoints REST d'authentification

Service métier

1
2
3
4
5
6
7
8
9
10
11
12
13
14
15
16
17
18
19
20
21
22
23
24
25
26
27
28
29
30
31
32
33
34
35
36
37
38
39
40
41
42
43

Listing 7 – AuthService.java — Logique métier d'authentification

Utilitaire JWT

1
2
3
4
5
6
7
8
9
10
11
12
13
14
15
16
17
18
19
20
21
22
23
24
25
26
27

Listing 8 – JwtUtil.java — Génération et validation des jetons JWT

3.4.4 Account Service

Le service de gestion des comptes encapsule toute la logique CRUD des comptes bancaires.

Entité Account

1
2
3
4
5
6
7
8
9
10
11
12
13
14
15
16
17
18
19
20
21
22

23
24
25
26
27
28
29
30
31
32

Listing 9 – Account.java — Entité JPA représentant un compte bancaire

Service métier

1
2
3
4
5
6
7
8
9
10
11
12
13
14
15
16
17
18
19
20
21
22
23
24
25
26
27
28
29
30
31
32
33
34
35
36
37
38
39
40
41
42
43
44

45
46
47
48
49
50
51
52
53
54
55
56

Listing 10 – AccountService.java — Logique métier de gestion des comptes (extrait)

3.4.5 Transaction Service

Le service de transactions est le plus complexe : il orchestre les appels inter-services via OpenFeign.

Clients Feign Les clients Feign déclarent les interfaces des services distants de manière déclarative. La résolution d'adresse passe par Eureka grâce à l'attribut `du`.

1
2
3
4
5
6
7
8
9
10
11

Listing 11 – AccountClient.java — Client Feign pour le account-service

1
2
3
4
5

Listing 12 – NotificationClient.java — Client Feign pour le notification-service

Entité Transaction

1
2
3
4
5
6
7
8

```
9
10
11
12
13
14
15
16
17
18
19
20
21
22
23
24
25
26
27
28
29
```

Listing 13 – Transaction.java — Entité JPA

Service métier — Virement

```
1
2
3
4
5
6
7
8
9
10
11
12
13
14
15
16
17
18
19
20
21
22
23
24
25
26
27
28
29
30
31
32
33
```

34
35
36
37
38
39
40
41
42
43
44
45
46
47
48
49
50
51
52
53
54
55
56
57
58
59
60
61
62
63
64
65
66
67
68

Listing 14 – TransactionService.java — Logique du virement (extrait)

3.4.6 Notification Service

1
2
3
4
5
6
7
8
9
10
11
12
13
14
15
16
17

18
19
20
21
22
23
24
25
26
27
28
29
30
31
32
33
34
35
36

Listing 15 – NotificationService.java — Journalisation des événements

3.4.7 Conteneurisation — Dockerfile

Tous les services utilisent le même multi-étapes :

1
2
3
4
5
6
7
8
9
10
11
12
13
14

Listing 16 – Dockerfile commun à tous les services

Ce multi-étapes (*multi-stage build*) présente deux avantages majeurs : (1) la séparation entre l'environnement de compilation et l'environnement d'exécution, et (2) la réduction significative de la taille de l'image finale en utilisant Alpine Linux.

3.4.8 Orchestration — Docker Compose

1
2
3
4
5
6

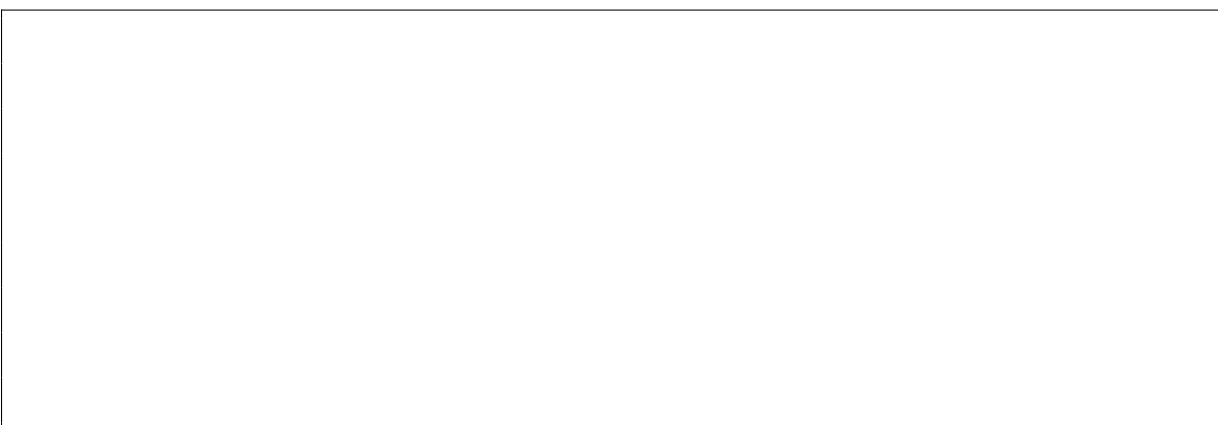
7
8
9
10
11
12
13
14
15
16
17
18
19
20
21
22
23
24
25
26
27
28
29
30
31
32
33
34
35
36
37
38
39
40
41
42
43
44
45
46
47
48
49
50
51
52
53
54
55
56
57
58
59
60
61
62
63
64
65

66
67
68
69
70
71
72

Listing 17 – docker-compose.yml — Orchestration de tous les services

3.4.9 Instructions de lancement

Pour lancer l'application complète, les commandes suivantes suffisent :



L'ordre de démarrage est géré automatiquement par les directives combinées aux . Le démarre en premier, puis les services métier, et enfin le (qui dépend de et). Le temps de démarrage initial est d'environ 3 à 5 minutes, car Maven télécharge les dépendances lors du premier build.

4 Partie C — Avantages et inconvénients

4.1 Avantages

4.1.1 Scalabilité indépendante

Chaque microservice peut être mis à l'échelle indépendamment en fonction de sa charge. Par exemple, si le reçoit un pic de requêtes lors des jours de paie, il est possible de déployer plusieurs instances de ce service sans dupliquer les autres. Docker Compose, et a fortiori Kubernetes, permettent cette scalabilité horizontale granulaire [2].

Illustration dans notre application : Le peut être répliqué horizontalement, et Eureka distribue la charge via le load balancing côté client ().

4.1.2 Déploiement continu facilité

L'indépendance des services permet de déployer des correctifs ou des nouvelles fonctionnalités sur un service sans affecter les autres. Cette caractéristique s'aligne parfaitement avec les pratiques CI/CD de la méthode DevOps. Un bug dans le peut être corrigé et redéployé sans interrompre les opérations bancaires [6].

Illustration : Chaque service possède son propre et peut être reconstruit et redéployé indépendamment via .

4.1.3 Isolation des pannes

Dans une architecture monolithique, une erreur dans un module peut entraîner l'arrêt complet de l'application. Avec les microservices, une panne d'un service n'affecte pas directement les autres. Dans notre application, si le tombe en panne, les transactions continuent de fonctionner grâce au bloc dans la méthode du [7].

Illustration : Le code du listing 14 montre que l'envoi de notification est enveloppé dans un . Un échec de notification génère un avertissement dans les logs mais n'empêche pas la transaction.

4.1.4 Flexibilité technologique

Chaque service peut utiliser la technologie la plus adaptée à son domaine. Bien que notre application utilise uniformément Java et Spring Boot, rien n'empêcherait de réécrire le en Python ou Go si les besoins l'exigeaient, tant que l'interface REST est respectée [1].

4.1.5 Équipes autonomes

Les microservices facilitent l'organisation en équipes pluridisciplinaires, chacune responsable d'un service de bout en bout (développement, tests, déploiement, monitoring). Cette organisation, inspirée de la loi de Conway, réduit les frictions inter-équipes et accélère le développement [9].

Illustration : Quatre équipes pourraient travailler simultanément sur les quatre services métier (auth, account, transaction, notification) sans se bloquer mutuellement.

4.1.6 Maintenabilité et compréhensibilité

Chaque microservice a une taille réduite et une responsabilité clairement définie, ce qui facilite la compréhension du code et réduit la charge cognitive des développeurs. Le principe de responsabilité unique (SRP) est naturellement respecté à l'échelle des services [8].

Illustration : Le ne contient que 66 lignes dans sa classe de service, ce qui le rend facilement compréhensible par un nouveau développeur.

4.2 Inconvénients

4.2.1 Complexité opérationnelle

L'architecture microservices introduit une complexité opérationnelle significative. Au lieu de gérer un seul déploiement, il faut orchestrer six conteneurs avec leurs dépendances de démarrage, leurs healthchecks, et leur réseau interne. Le fichier de notre application fait déjà 119 lignes pour six services relativement simples [3].

4.2.2 Latence réseau inter-services

Contrairement à un monolithe où les appels entre modules sont des appels de méthodes en mémoire (nanosecondes), les appels inter-services passent par le réseau (millisecondes). Un virement dans notre application nécessite au minimum six appels réseau (deux lectures de compte, deux mises à jour de solde, deux notifications), ce qui multiplie la latence totale.

4.2.3 Gestion de la cohérence des données

Le principe *database per service* rend difficile le maintien de la cohérence des données entre services. Dans notre application, un virement qui échoue après le débit du compte source mais avant le crédit du compte cible pourrait laisser le système dans un état incohérent. Des patrons comme le *Saga Pattern* ou l'*Event Sourcing* seraient nécessaires en production pour gérer ces cas [3].

4.2.4 Coût d'infrastructure

Chaque microservice nécessite ses propres ressources (mémoire, CPU, stockage). Six instances JVM Spring Boot consomment significativement plus de ressources qu'une seule application monolithique équivalente. Le coût en infrastructure peut être prohibitif pour des petites équipes ou des startups en phase initiale.

4.2.5 Débogage difficile

Tracer une requête à travers plusieurs services est considérablement plus complexe que dans un monolithe. Sans outils de traçage distribué comme Zipkin ou Jaeger, il est difficile de suivre le cheminement d'une requête et d'identifier la source d'une erreur. Notre application s'appuie sur les logs de chaque service, ce qui peut s'avérer insuffisant dans un environnement de production à grande échelle.

4.2.6 Surcharge pour les petits projets

Pour des projets de petite taille ou des prototypes, l'architecture microservices est souvent disproportionnée. La complexité ajoutée (réseau, découverte de services, gestion des pannes

distribuées) n'est pas justifiée si le système peut tenir dans un monolithe bien structuré. Martin Fowler recommande de commencer par un monolithe et de migrer vers les microservices uniquement lorsque la complexité le justifie [5].

4.3 Quand utiliser les microservices ?

4.3.1 Cas favorables

L'architecture microservices est recommandée dans les situations suivantes :

- **Applications à grande échelle** nécessitant une scalabilité différenciée par composant ;
- **Équipes nombreuses** travaillant en parallèle sur des domaines fonctionnels distincts ;
- **Systèmes critiques** nécessitant une haute disponibilité et une isolation des pannes ;
- **Organisations pratiquant le DevOps** avec des pipelines CI/CD matures ;
- **Applications à évolution rapide** où des fonctionnalités doivent être déployées fréquemment et indépendamment.

4.3.2 Cas défavorables

L'architecture microservices est déconseillée dans les cas suivants :

- **Petites applications** avec peu de domaines fonctionnels ;
- **Équipes réduites** (moins de 5 développeurs) sans expertise DevOps ;
- **Prototypes ou MVP** où la rapidité de développement prime ;
- **Systèmes à forte cohérence transactionnelle** nécessitant des transactions ACID inter-domaines ;
- **Budget d'infrastructure limité.**

4.3.3 Recommandations

À la lumière de notre expérience avec ce projet et de la littérature académique, nous formulons les recommandations suivantes :

1. **Commencer simple** : démarrer par un monolithe bien structuré en couches, puis extraire les microservices lorsque la complexité le justifie (*Monolith First* [5]) ;
2. **Investir dans l'outillage** : l'adoption des microservices doit s'accompagner d'outils de monitoring (Prometheus, Grafana), de traçage distribué (Zipkin, Jaeger) et de gestion des logs centralisée (ELK Stack) ;
3. **Définir des interfaces claires** : les contrats d'API entre services doivent être documentés et versionnés (OpenAPI/Swagger) pour minimiser le couplage ;
4. **Automatiser tout** : CI/CD, tests, déploiement, monitoring — l'automatisation est la clé de la réussite d'une architecture microservices dans l'esprit DevOps.

5 Conclusion

Ce travail nous a permis d'explorer en profondeur l'architecture microservices, tant sur le plan théorique que pratique. À travers la conception et l'implémentation d'une application bancaire distribuée en six microservices, nous avons pu expérimenter concrètement les principes fondamentaux de cette approche architecturale.

Sur le plan théorique, nous avons constaté que l'architecture microservices incarne les principes clés étudiés dans le cours d'architecture des applications d'entreprise :

- La **modularité** se manifeste par la décomposition de l'application en services autonomes, chacun encapsulant un domaine métier cohérent ;
- Le **couplage faible** est assuré par la communication via des API REST bien définies et la découverte dynamique des services via Eureka ;
- Les **couches architecturales** sont respectées au sein de chaque service (Controller, Service, Repository, Model), conformément au patron *Layered Architecture* ;
- Les **patrons architecturaux** comme l'API Gateway, le Service Registry, et le Database per Service illustrent des solutions éprouvées aux problèmes récurrents des systèmes distribués.

Sur le plan pratique, la réalisation de ce projet nous a confrontés aux défis réels de l'architecture microservices : la gestion des dépendances de démarrage entre services, la communication inter-services fiable via OpenFeign, la conteneurisation avec Docker, et l'orchestration avec Docker Compose. Ces défis nous ont permis de comprendre pourquoi l'adoption des microservices doit être une décision mûrement réfléchie, en fonction de la taille du projet, de l'équipe et des besoins de scalabilité.

En définitive, l'architecture microservices, couplée à la méthode DevOps, représente une approche puissante pour le développement d'applications d'entreprise modernes. Cependant, elle n'est pas une solution universelle et doit être adoptée en connaissance de ses implications en termes de complexité opérationnelle et de coûts d'infrastructure.

6 Références bibliographiques

Références

- [1] Fowler, M. et Lewis, J. (2014). *Microservices : a definition of this new architectural term*. martinowler.com. Repéré à
- [2] Newman, S. (2021). *Building Microservices : Designing Fine-Grained Systems* (2e éd.). O'Reilly Media.
- [3] Richardson, C. (2018). *Microservices Patterns : With Examples in Java*. Manning Publications.
- [4] Evans, E. (2003). *Domain-Driven Design : Tackling Complexity in the Heart of Software*. Addison-Wesley.
- [5] Fowler, M. (2015). *MonolithFirst*. martinowler.com. Repéré à
- [6] Humble, J. et Farley, D. (2010). *Continuous Delivery : Reliable Software Releases through Build, Test, and Deployment Automation*. Addison-Wesley.
- [7] Nygard, M. T. (2018). *Release It! : Design and Deploy Production-Ready Software* (2e éd.). Pragmatic Bookshelf.
- [8] Martin, R. C. (2017). *Clean Architecture : A Craftsman's Guide to Software Structure and Design*. Prentice Hall.
- [9] Conway, M. E. (1968). How do committees invent ? *Datamation*, 14(4), 28–31.
- [10] Spring Cloud. (2024). *Spring Cloud Documentation*. Repéré à